

86. Trois variables qui parlent de la même chose

Une `struct`, plutôt que `bossX`, `bossY`, `bossVie`.

1. Ce qu'il faut comprendre

Sans `struct`, on écrirait `bossX`, `bossY`, `bossVie` — et le jour où il faut un second ennemi, on recopie trois variables et l'on en oublie une.

Une `struct` fait **au plus 255 octets**, et chaque champ est un octet. C'est un enregistrement, pas un objet : ni héritage, ni constructeur.

La leçon « Une struct : ce qui va ensemble » en fait un usage complet ; celle-ci n'en montre que la forme.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **531 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         66 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   67 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         224 o ← CopierTuiles - 14 lettres sur 43
"X"            1 o ← texte() - un texte gravé
"VIE"          3 o ← texte() - un texte gravé
```

Lettres gravées : EIVX0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 2   22 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : range la place et les vies d'un ennemi ENSEMBLE, dans une struct,
// puis les affiche.
//
// struct Ennemi ...;   = on invente un type « Ennemi » qui contient 3 cases : x, y, vie.
// Ennemi boss;         = on crée UN ennemi, nommé boss.
// boss.x               = la case x DE boss (le point veut dire « de »).

#include <texte>         // écrit un texte à l'écran
#include <nombre>       // écrit un nombre en chiffres
```

2. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {           // Le modèle d'un ennemi :
    uint8_t x;             // sa colonne en pixels,
    uint8_t y;             // sa ligne en pixels,
    uint8_t vie;           // ses points de vie.
};                         // (le point-virgule après l'accolade est obligatoire)
```

3. UNE VARIABLE — BOSS

```
Ennemi boss;              // Un ennemi, nommé boss.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

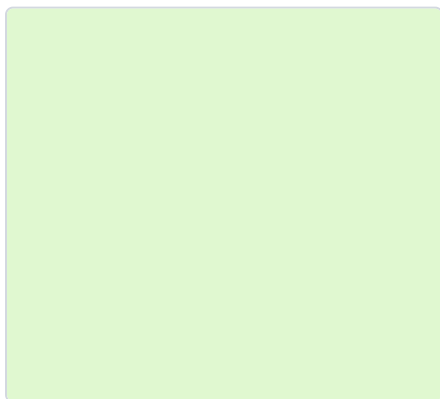
```
int main() {              // Le jeu commence ici.
    boss.x = 40;           // On remplit ses trois cases.
    boss.y = 60;
    boss.vie = 3;

    texte(2, 4, "X");      nombre(6, 4, boss.x);      // Affiche 040.
    texte(2, 6, "VIE");   nombre(6, 6, boss.vie, 1); // Affiche 3.

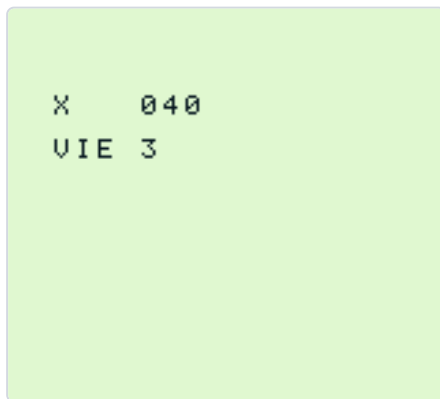
    while (true) {         // Répète pour toujours :
        image();           // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« X 040 » et « VIE 3 » : les deux champs de la même struct, relus.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le champ `x` est écrit et relu
- ✓ le champ `vie` aussi
- ✓ les trois champs tiennent en trois octets

Ensuite : **87. Un tableau de « struct »** — chapitre 13 : Ce qui va ensemble. · [gameboy3](#) — écrire une cartouche Game Boy en C++